

Hybrid Parsing for High-Performance and Extensible Systems

1.0 Introduction

Modern software systems increasingly rely on complex build pipelines, code analysis tools, and configuration-driven workflows. These systems often require the ability to process both structured data formats and full programming languages in a unified and efficient manner.

Typical use cases include dependency resolution, project classification, linting, and transformation of intermediate representations.

In such environments, parsing is no longer a one-off compilation step, but a core component of the runtime toolchain. It must operate efficiently across large codebases, support multiple input formats, and remain flexible enough to handle language-specific edge cases.

A representative scenario arises in convention-over-configuration build systems, where projects are analyzed based on their public interfaces, dependencies, and structural metadata. These systems must:

- Parse structured configuration formats such as JSON for build targets and external dependencies
- Process high-level programming languages to extract semantic information

- Classify projects automatically (e.g., application vs. dependency) using linting rules
- Resolve and validate dependency graphs across multiple projects

These requirements introduce a combination of constraints that are difficult to satisfy with traditional parsing approaches:

- High throughput is required to minimize overall build time
- High accuracy is necessary to ensure correctness of dependency resolution
- Multiple languages and formats must be supported within a unified framework
- The system must remain extensible to accommodate language-specific constructs such as macros or conditional compilation

Conventional parser generators, such as table-driven LR parsers or generalized parsing frameworks, provide strong theoretical guarantees but often introduce practical limitations.

They tend to produce large, opaque parsing tables, offer limited control over execution, and make it difficult to integrate custom logic for non-context-free features.

At the other end, fully hand-written parsers provide maximum control and performance but suffer from reduced maintainability and limited reuse. Each new language or format typically requires significant manual effort, and shared optimizations are difficult to generalize.

This work presents a hybrid parsing framework that combines the strengths of both approaches. The system is inspired by LR(n) parsing principles but replaces traditional table-driven execution with an explicit, state-based model that allows fine-grained control over transitions and execution flow.

The key idea is to unify generated grammar rules and manually implemented logic within a single deterministic state machine. Grammar fragments can be defined using a compact domain-specific language (DSL) and compiled into optimized state transitions, while complex or context-dependent behavior can be implemented as custom passes integrated directly into the same execution model.

This hybrid approach enables:

- Deterministic parsing behavior comparable to LR(n) systems
- High-performance execution through switch-based state machines and streaming input processing
- Seamless integration of custom logic for cases that exceed context-free grammar expressiveness
- Reusability across different domains, including configuration parsing, preprocessing, and static analysis

The following sections describe the design goals, internal architecture, and rule compilation model of the framework, along with its application in modern high-performance tooling environments

2.0 Design Goals and Constraints

The design of the parsing framework is guided by a set of practical constraints derived from real-world tooling scenarios. These constraints emphasize performance, determinism, and extensibility while maintaining a minimal and predictable execution model

2.1 Deterministic Execution

The parser is designed to operate without backtracking. All parsing decisions are made deterministically based on the current state and a bounded amount of lookahead. This behavior is conceptually aligned with LR(n) parsing, where decisions are derived from a fixed context of upcoming tokens.

Deterministic execution ensures predictable performance characteristics and avoids exponential behavior commonly associated with backtracking parsers

2.2 Streaming Input Processing

Input is processed in a streaming fashion, allowing the parser to operate on large inputs without requiring full materialization in memory. This is particularly important for large codebases and build pipelines where input size can be substantial.

Streaming also enables incremental processing and reduces memory pressure by avoiding unnecessary allocations

2.3 Performance and Memory Locality

The framework prioritizes linear execution patterns, minimal branching, and

cache-friendly data access. Both tokenization and parsing are implemented as state machines using direct control flow constructs (e.g., switch-based dispatch), avoiding indirect table lookups where possible.

Buffers for text and tokens are reused and managed explicitly to minimize allocations and improve locality

2.4 Extensibility Through Hybrid Execution

While many language constructs can be expressed using context-free grammars, real-world systems often require additional logic. Examples include macro expansion, conditional compilation, and context-sensitive validation.

To address this, the framework allows manual passes to be integrated directly into the parsing process. These passes operate within the same state-based execution model and can manipulate token streams, introduce sub-streams, or alter control flow in a controlled and deterministic manner

2.5 Multi-Language and Multi-Format Support

The framework is intended to support a wide range of input formats, including structured data (e.g., JSON) and full programming languages. This requires a flexible rule definition mechanism and a shared execution model that can accommodate different grammar complexities

2.6 Error Handling and Recovery

Rather than terminating on the first error, the parser collects errors during execution

and attempts to recover by returning to previously valid states. This approach is essential for tooling scenarios such as linting and static analysis, where partial results are often more valuable than complete failure.

Error handling is integrated into the state machine, allowing recovery strategies to be applied consistently across both generated and manually implemented rules

3.0 Core Parsing Model

The framework is built around a streaming, state-driven execution model that unifies tokenization and parsing into a predictable and efficient pipeline. Both stages operate on shared principles: explicit state machines, controlled buffering, and deterministic transitions

3.1 Streaming-Oriented Input Processing

Input is processed incrementally rather than materialized in full. The tokenizer consumes characters from an input stream and produces tokens on demand. The parser, in turn, consumes tokens as they become available. This approach enables processing of large inputs with minimal memory overhead and allows the system to interleave tokenization and parsing without intermediate storage

3.2 Text Buffer and Input Handling

Character input is accumulated in a reusable text buffer. The buffer exposes operations such as peeking ahead, trimming consumed prefixes, and clearing intermediate content. This allows efficient

construction of token values without repeated allocations.

Optionally, the system maintains a caret (line and column) that is updated when a token is finalized. Once a token pass completes, a finalize step resolves line breaks within the buffer and advances the caret accordingly. This ensures accurate position tracking for diagnostics without impacting the hot path of character consumption

3.3 Tokenization as a State Machine

Tokenization is implemented as a deterministic state machine. Each pass attempts to recognize a single token from the current input position. The implementation follows a switch-based control flow, where states correspond to partial matches of token patterns.

A token pass either:

- Produces a valid token in the buffer and finalizes it, or
- Terminates with an error if no valid token can be formed

Because tokenization operates incrementally and deterministically, it can be tightly integrated with the parser without requiring a full token list

3.4 Token Stream and Buffering

Tokens are not stored in a precomputed sequence. Instead, a single token buffer is reused, and tokens are produced on demand. The parser requests the next token when required, triggering a tokenization pass that fills the buffer.

This design avoids redundant storage and allows lookahead to be implemented explicitly by controlling when new tokens are requested

3.5 Parser Execution Model

The parser is implemented as a state machine similar to the tokenizer. It operates on tokens provided by the token buffer and advances through grammar rules using explicit state transitions.

Execution follows a switch-based dispatch on the current state. Each state evaluates the current token and determines the next action

3.6 State Stack and Rule Progression

Parsing is driven by a stack of active states. Each stack entry represents both:

- The current rule being evaluated
- The current position (state) within that rule

This allows the parser to track nested rule invocations and partial progress within each rule. When a rule references another fragment or sub-rule, a new state is pushed onto the stack. Upon completion, control returns to the previous state.

The execution flow can be summarized as:

- Switch on current parser state
- Evaluate rule context
- Switch on rule-internal state
- Evaluate current token

- Perform a transition and update the stack accordingly

This explicit representation replaces traditional LR parsing tables while preserving deterministic behavior and fine-grained control over execution

3.7 State Transitions

Each state transition encodes a specific operation. Transitions are not limited to simple state changes; they define how the parser interacts with the token stream and the state stack.

Multiple transition flags may be combined to express complex behavior.

Core transition operations include:

- **Next:** Consume the current token while keeping the current stack state
- **Shift:** Consume the current token and advance the rule state
- **Reduce:** Pop the current state from the stack and return to the previous rule
- **Revert:** Reset the stack to its initial state
- **Success:** Reset the stack and signal successful completion of the current pass
- **Error:** Reset the stack and signal a parsing error
- **Preserve:** Do not consume the current token

By combining these operations, the parser can express deterministic control flow without backtracking. For example, a transition may both advance the rule state and preserve the token for re-evaluation in a different context

3.8 Integrated Token–Parser Interaction

Tokenization and parsing are interleaved during execution:

- The tokenizer reads from the input stream
- A token pass produces a valid token or signals an error
- The parser consumes the token from the buffer
- The parser evaluates the token and performs a state transition
- If additional input is required, the parser requests the next token, triggering another tokenization pass

This cycle continues until the parser reaches a success or error condition. As rules progress, new states may be pushed or popped, and fragments may be constructed or discarded depending on the outcome

3.9 Error Handling and Recovery (Overview)

Errors may be collected during parsing rather than causing immediate termination if possible. When an error occurs, the parser can try to reset or adjust the state stack and attempt to resume execution from a previously valid configuration.

This behavior allows partial results to be produced and enables use cases such as linting, static analysis or complex language processing, where complete correctness is less important than maximal coverage

4.0 State-Based LR(n) Hybrid Execution

Traditional LR parsers rely on precomputed parsing tables that encode all possible state transitions for a given grammar. While this approach provides strong guarantees regarding determinism and correctness, it introduces significant complexity in both generation and execution. The resulting tables are often large, opaque, and difficult to adapt when additional, non-context-free behavior is required.

The approach presented in this work follows the core principle of LR(n) parsing—deterministic decision making based on a bounded lookahead—but replaces table-driven execution with an explicit, state-based model

4.1 From Table-Driven to Explicit State Machines

In classical LR parsing, the parser maintains a stack and consults parsing tables to decide whether to shift, reduce, or accept based on the current state and lookahead token. These tables are typically generated from a grammar and are not intended to be modified manually.

In contrast, the presented system encodes these decisions directly in executable control flow. Each parser state corresponds to a concrete code path, and transitions are expressed through explicit operations rather than table lookups.

This transformation has several implications:

- Parsing logic becomes transparent and directly inspectable

- Control flow can be optimized using standard compiler techniques
- Manual extensions can be integrated without breaking the model

4.2 LR(n) Characteristics Without Tables

Although the implementation differs, the system preserves key properties associated with LR(n) parsing:

- Decisions are made deterministically
- A bounded amount of lookahead is sufficient for correct transitions
- The parser maintains an explicit stack representing nested rule contexts

Rather than encoding lookahead behavior in tables, the system expresses it through state transitions and controlled token consumption. When necessary, multiple tokens may be inspected by deferring consumption using the Preserve transition

4.3 Deterministic Transition Model

Determinism is achieved by ensuring that, for any given state and token, the next transition is unambiguous. This is enforced during rule generation and optimization, where overlapping paths are merged and ambiguous branches are resolved.

As a result, the parser avoids backtracking entirely. All execution paths are linear and predictable, which is critical for high-performance scenarios

4.4 Fine-Grained Control Over Execution

Because transitions are encoded as explicit operations, the system provides fine-grained control over parsing behavior. Each transition can:

- Consume or preserve tokens
- Advance or rewind rule states
- Modify the state stack
- Signal success or failure conditions

This level of control allows the parser to handle cases that are difficult to express in pure context-free grammars, while still maintaining a structured and deterministic execution model

4.5 Controlled Lookahead and Re-evaluation

Unlike traditional LR parsers, where lookahead is implicitly encoded in tables, this system allows lookahead to be expressed explicitly. Tokens can be preserved and re-evaluated in different states, enabling controlled multi-token inspection without introducing backtracking.

This mechanism is particularly useful for constructs where the correct interpretation depends on more than one token, such as macro invocations or context-sensitive expressions

4.6 Integration with Hybrid Parsing

The explicit state model also serves as the foundation for integrating manually implemented passes. Since both generated rules and custom logic operate on the same

state machine, they can interact seamlessly.

Manual passes can:

- Consume or inject tokens
- Manipulate the state stack
- Trigger transitions directly

Despite this flexibility, the overall system remains deterministic, as all operations must resolve to well-defined state transitions

5.0 Hybrid Parsing: Generated and Manual Passes

While context-free grammars provide a solid foundation for parsing, real-world languages and tooling scenarios frequently require behavior that exceeds their expressive capabilities. Features such as macro expansion, conditional compilation, and context-sensitive interpretation introduce dependencies that cannot be modeled purely through static grammar rules.

The presented framework addresses these limitations through a hybrid parsing model that combines generated grammar rules with manually implemented passes, both operating within a unified execution model

5.1 Limitations of Pure Grammar-Based Parsing

Traditional parser generators assume that the structure of a language can be fully described using a context-free grammar. While this assumption holds for many core

language constructs, it breaks down in the presence of:

- Macro systems that transform token streams dynamically
- Conditional compilation directives that alter control flow based on evaluated expressions
- Context-sensitive constructs where interpretation depends on semantic information

In such cases, purely generated parsers either become overly complex or require external preprocessing stages, which fragment the parsing pipeline and reduce control over execution

5.2 Motivation for a Hybrid Model

To address these challenges, parsing must remain both deterministic and extensible. A hybrid model allows grammar-based structure to coexist with targeted, manually implemented logic for complex scenarios.

The key requirement is that this integration does not compromise determinism or introduce hidden execution paths. All behavior—whether generated or manual—must remain part of a single, controlled system

5.3 Unified State Machine Model

The core of the hybrid approach is a unified state machine. Both generated grammar rules and manual passes are represented as states within the same execution model.

Each state is identified by a unique identifier that maps directly to a concrete execution point in the parser. Conceptually, this can be understood as a switch-based dispatch

mechanism, where state identifiers act as jump targets within a single control flow structure.

In this model:

- Generated rules compile into state identifiers with predefined transitions
- Manual logic is attached to specific state identifiers as custom handlers
- All execution paths are resolved through the same switch-based dispatch

The generator is responsible for producing the structural layout of these states, including the transition graph and entry points. At designated states, extension points are created where custom logic can be inserted.

These extension points follow a uniform method signature, ensuring that custom implementations integrate seamlessly into the execution model while leaving the actual behavior under full developer control.

As a result, there is no distinction at runtime between generated and manual execution paths. Both are compiled into the same state machine and participate equally in deterministic transitions

5.4 Custom Pass Integration

Custom Pass Integration

Manual passes are integrated directly into the state machine and can be invoked from any state. These passes are not treated as external callbacks but as first-class participants in the parsing process.

A custom pass may:

- Consume tokens from the current stream
- Produce or replace tokens
- Introduce sub-streams for localized parsing
- Manipulate the parser state stack

Example:

```
STATE_MACRO_EXPANSION:
  call ExpandMacro()
  transition NEXT
```

After execution, control returns to the parser with a well-defined state, ensuring that the overall system remains deterministic

5.5 Token Stream Manipulation and Conditional Evaluation

One of the most important capabilities enabled by the hybrid model is controlled manipulation of the token stream through custom logic.

Macro expansion is a primary example: an identifier token may be replaced with an entire sequence of tokens, which must then be reprocessed by the parser. This is achieved by inserting a sub-stream into the token flow and resuming parsing without restarting the system.

Custom passes may also implement higher-level logic such as conditional evaluation. In such cases, a sequence of tokens is interpreted as an expression, evaluated according to operator precedence, and reduced to a boolean result.

This introduces a temporary transition from syntactic parsing to a custom evaluation phase:

- Tokens are consumed and interpreted as an operator-based expression
- The expression is evaluated to a boolean value
- Control returns to the parser with a deterministic outcome

It is important to note that the framework itself only provides the mechanisms—state transitions, token control, and stack manipulation—that allow such behavior to be implemented as part of custom logic.

This design keeps the core parsing model minimal while enabling complex, domain-specific behavior where required

5.6 Determinism in Hybrid Execution

Determinism in Hybrid Execution

Despite the flexibility introduced by manual passes, the system maintains strict determinism.

Each state—whether generated or manual—must resolve to a single, well-defined outcome. There are no implicit branches or hidden backtracking paths. All transitions are explicit, and all token consumption is controlled.

This ensures that the hybrid model retains the performance characteristics of LR-based parsing while extending its expressive power

5.7 Interaction Between Generated and Manual Rules

Generated rules and manual passes are not layered on top of each other but interwoven within the same execution graph. A generated rule may invoke a manual pass, and a manual pass may return control to a generated rule.

This bidirectional interaction enables seamless handling of complex language features without introducing separate processing stages

5.8 Context-Sensitive Interpretation

Certain constructs cannot be resolved based on syntax alone and require contextual interpretation. For example, ambiguous expressions may depend on whether a token represents a type, a variable, or an operator.

In such cases, the parser can transition into a custom pass that evaluates the context and determines the correct interpretation. The result is then mapped back into the state machine as a deterministic transition.

This approach allows the system to handle context-sensitive parsing scenarios without abandoning its structured execution model

6.0 Rule Definition Language (DSL)

To support the hybrid parsing model described in the previous sections, a dedicated rule definition language (DSL) is introduced. The DSL is designed to be minimal, self-descriptive, and directly

translatable into the unified state machine model.

Unlike traditional grammar specifications, the DSL separates syntactic structure from generator-specific semantics through the use of constant groups and directional binding

6.1 Design Goals

The DSL is defined with the following objectives:

Provide a compact and expressive way to describe grammar rules

- Remain fully self-descriptive
- Avoid reliance on layout (e.g., line breaks or indentation)
- Integrate seamlessly with generated and manual parsing logic
- Allow extension through generator-defined constants without modifying the core grammar

6.2 Core Grammar

The DSL grammar is defined as follows:

Rule = Context* PlainRule Transition*;

!fragment PlainRule = Identifier BeginRule Statement+ EndRule;

Group = BeginGroup Statement* EndGroup;

!fragment **Statement** = Identifier | Literal | Range | Group | (Statement OrSequencer Statement) | UnaryStatement;

!fragment **UnaryStatement** = (InvertQuantifier Statement) | (Statement Sequencer);

Sequencer = OptionalSequencer | AnyQuantifier | IterationQuantifier;

Range = BeginRange Literal RangeSeparator Literal EndRange;

Tokens are defined as follows:

Comment = SingleLineComment | MultiLineComment; *!discard*

!fragment **SingleLineComment** = *//* (~\n)*;

!fragment **MultiLineComment** = */** (~**/*)* */**?;

Identifier = ([a',z'] | [A',Z'])+;

Literal = SingleQuotationLiteral | DoubleQuotationLiteral;

!fragment **SingleQuotationLiteral** = *"* (\\" | ~\")* *"*;

!fragment **DoubleQuotationLiteral** = *'''* (\\" | ~''')* *'''*;

BeginRule = '=';
BeginGroup = '(';
BeginRange = '[';
EndRange = ']';
EndGroup = ')';
EndRule = ';';
OrSequencer = '|';

OptionalSequencer = '?';

AnyQuantifier = '*';

IterationQuantifier = '+';

InvertQuantifier = '~';

RangeSeparator = ',';

Additionally, the DSL introduces symbolic constant groups as follows:

Context = CONTEXT;

Transition = TRANSITION;

Whitespace = WHITESPACE+; *!discard*

The grammar is self-descriptive and can be parsed using the same mechanisms it defines

6.3 Constant Groups

The DSL introduces symbolic constant groups that are resolved by the generator. These groups define extensible sets of flags that influence rule generation and execution behavior.

The primary groups are:

- CONTEXT
- TRANSITION
- WHITESPACE

Each group represents a set of valid constants. Concrete values, such as *!fragment*, *!extern*, or *!discard*, are elements of these sets and are interpreted by the generator.

The DSL itself does not define the contents of these groups. Instead, they are provided externally and may vary depending on the target system

6.4 Directional Binding of Constants

Constants are not interpreted based on textual layout or whitespace, but through their relation to recognized rules in the token stream.

Binding is defined directionally:

- Context constants are associated with the subsequent rule
- Transition constants are associated with the preceding rule

This allows constants to be placed independently of formatting while preserving unambiguous semantics.

Example:

!discard !fragment **Rule** = A B C; *!extern*

is interpreted as:

- *!discard* → transition of this rule's predecessor
- *!fragment* → context of Rule
- *!extern* → context of the next rule

Invalid placements are well-defined:

- A transition constant without a preceding rule results in an error
- A context constant without a subsequent rule results in an error

6.5 Separation of Syntax and Semantics

A key property of the DSL is the strict separation between grammar definition and execution semantics.

- The grammar defines structure (rules, expressions, tokens)
- Constant groups define generator-level behavior

For example:

- *!fragment* influences whether a rule produces a distinct state identifier
- *!extern* introduces an entry point for custom logic
- *!discard* modifies the behavior of successful transitions

These behaviors are not part of the grammar itself but are applied during code generation

6.6 Self-Descriptive Capability

The DSL is capable of describing its own grammar. This property ensures that:

- No external grammar definition is required
- The parser can be generated from the DSL itself
- The system remains internally consistent

This recursive definition reinforces the uniformity of the parsing model and simplifies tooling

6.7 Mapping to the Execution Model

Each rule defined in the DSL is translated into one or more states within the unified state machine described in previous chapters.

- Identifiers map to state entry points
- Statements define transitions between states
- Context constants influence state generation
- Transition constants define completion behavior

Custom logic can be attached to generated states through externally defined hooks, enabling seamless integration between generated and manual parsing logic

6.8 Token and Rule Classification (Syntactic Distinction)

In addition to semantic classification via generator constants, the DSL defines a syntactic distinction between tokens and rules based on structural properties of their definitions.

A definition is considered a rule if it introduces structural composition beyond terminal expansion. This is typically indicated by the presence of a PlainRule construct, which follows the form:

Identifier = *Statement***+**;

Rules are characterized by:

- The presence of a = assignment operator
- A sequence of one or more Statement elements
- Potential recursive composition through groups, sequencing, or alternatives

A definition is considered a token if it describes a terminal construct without structural expansion into higher-level rule composition. Tokens typically resolve directly to character-level patterns such as literals, ranges, or simple sequences of characters.

Fragments (*!fragment*) are syntactically rules, but are excluded from rule classification at the generation level. They are used purely for structural reuse and do not produce independent parser entry points.

This leads to a three-level syntactic model:

- Tokens: terminal definitions used directly by the lexer/parser
- Rules: compositional definitions that define parsing structure
- Fragments: structural helpers that are syntactically rules but semantically non-emitting

This distinction is derived from syntax alone and does not require semantic interpretation or reference analysis

7. Code Generation and State Machine Construction

This chapter describes the transformation of DSL-defined rules into an executable parser. The process is based on a sequence of normalization, expansion, and optimization steps that result in a deterministic state machine. Unlike traditional LR parser generators, ambiguities are resolved during generation rather than at runtime

7.1 Overview of the Generation Pipeline

The generation process consists of the following stages:

1. Rule analysis and normalization
2. Grammar expansion and conflict resolution
3. Fragment resolution
4. Path merging via graph construction
5. State machine generation

Each stage incrementally transforms the DSL into a form suitable for efficient execution

7.2 Rule Normalization

DSL rules may contain nested groups, alternations, and quantifiers. These constructs are normalized into linearized rule fragments.

Example:

Expression = *Term* ((*Plus* | *Minus*) *Term*)*;

is transformed into a sequence of explicit transitions:

Expression -> *Term ExpressionTail*
ExpressionTail -> (*Plus Term ExpressionTail*) | (*Minus Term ExpressionTail*) | ϵ

This step ensures that all rules can be processed as deterministic sequences

7.3 Grammar Expansion and Conflict Resolution

Ambiguities caused by shared prefixes are resolved through rule expansion. Consider the following definitions:

IncludeNamespace = *Using Identifier (Dot Identifier)* Semicolon*;

AliasDeclaration = *Using Identifier EqualSign Identifier (Dot Identifier)* Semicolon*;

UsingBlock = *Using BraceLeft Identifier EqualSign Identifier (Dot Identifier)* BraceRight*;

All rules share the prefix *Using*, and two of them share *Using Identifier*. This creates ambiguity in a classical LR(1) parser. Instead of increasing lookahead, the rules are expanded:

UsingStatement = *Using Identifier*
UsingIdentifierTail | *Using BraceLeft*
UsingBlockTail;

!fragment UsingIdentifierTail = (*Dot*
Identifier)* *Semicolon* | *EqualSign Identifier*
(*Dot Identifier*)* *Semicolon*;

This transformation produces a deterministic branching structure. Conflicts are therefore eliminated during generation rather than resolved at runtime

7.4 Fragment Resolution

Rules marked with **!fragment** are expanded inline and do not produce independent states. They are used to simplify rule definitions and reduce duplication.

Fragments are substituted into referencing rules prior to state generation

7.5 Token and Rule Materialization

After expansion, definitions are classified into tokens and rules.

- Tokens represent terminal symbols produced by the tokenizer
- Rules define parser-level structure

Example token definitions:

Plus = '+';
Minus = '-';
BraceLeft = '(';
BraceRight = ')';

Parser rules operate exclusively on these symbolic tokens

7.6 Path Merging via Acyclic Graph Construction

After expansion, multiple rule paths often share common prefixes. These paths are merged using an acyclic graph structure. The graph is constructed by inserting all possible token sequences into a shared structure:

```
function BuildGraph(paths):  
    root = Node()  
    for path in paths:  
        current = root  
        for element in path:  
            current = current.getOrCreate(element)  
    return root
```

This produces a prefix-sharing graph (similar to a trie), which is then used to generate code. Code generation traverses the graph recursively:

```
function Generate(node):  
    switch(next token):  
        for each edge in node:  
            case edge.value:  
                if edge.hasChildren:  
                    consume()  
                    Generate(edge.child)  
            else:  
                accept()
```

This approach reduces redundancy and minimizes the number of generated states

7.7 State Machine Generation

Each normalized rule is translated into a set of states. Each state represents a position within a rule.

Example pseudocode:

```
switch (state) {
  case STATE_EXPRESSION:
    if token == Identifier:
      transition SHIFT to STATE_TERM
    else:
      transition ERROR
}
```

States are identified by unique identifiers and are connected through transitions

7.8 Transition Construction

Transitions define how the parser reacts to tokens. Each transition is composed of flags:

- NEXT
- SHIFT
- REDUCE
- REVERT
- SUCCESS
- ERROR
- PRESERVE

These flags determine stack manipulation and token consumption behavior.

Example:

```
if token == Plus:
  transition SHIFT | REDUCE
```

SHIFT and NEXT are mutually exclusive operations. SHIFT advances the parser state, while NEXT preserves the current state and consumes the token. Combining SHIFT with REDUCE allows progression and consolidation of the current rule in a single step

7.9 Stack Integration

The parser maintains a stack of states. Each entry represents the current progress within a rule.

- SHIFT pushes a new state
- REDUCE pops the current state
- REVERT clears the stack

This enables nested rule evaluation and backtracking to valid states

7.10 Integration of Custom Logic

Rules marked with !extern generate explicit entry points for custom logic. These entry points act as anchors within the generated state machine and allow controlled transitions between generated and manually implemented logic.

The generator produces a state identifier and a corresponding hook, which can be implemented manually.

Example:

```
switch (state) {
  case STATE_USING:
    if token == Using:
      call extern anchor(Using)
      transition SHIFT to STATE_IDENTIFIER
    else
      transition ERROR
```

}

The external anchor serves as a synchronization point. It allows the developer to introduce additional logic such as tracking, validation, or context-sensitive processing without breaking the deterministic execution model.

If a branch becomes unambiguous after entering external logic, the parser may fully transition into the external implementation. Conversely, if ambiguity remains, the generator ensures that both generated and external paths are merged.

To support this, rules are internally split into sub-fragments. These fragments allow the generator to interleave external logic with generated transitions. Overlapping paths between generated and external logic are unified to prevent ambiguity.

This guarantees that:

- External logic does not introduce nondeterminism
- All transitions remain part of a single coherent state machine
- Shared prefixes between generated and custom logic are merged

This mechanism is essential for advanced scenarios such as macro expansion, conditional evaluation, or language-specific extensions.

A representative example can be found in compiler pipelines similar to Roslyn, where syntactic parsing and semantic analysis interact. Consider name binding or contextual keyword handling: a token sequence may be syntactically valid but

requires semantic context (e.g., symbol tables, scope resolution) to determine its meaning. An external anchor can be placed at the relevant rule boundary (e.g., after recognizing an identifier or a declaration prefix). The generated parser reaches the anchor, invokes external logic to perform symbol lookup or contextual classification, and then resumes within the same deterministic state machine.

Crucially, if multiple syntactic paths share prefixes with such context-sensitive constructs, the generator merges these paths and inserts the external anchor at the shared point. Internally, the affected rules are split into sub-fragments so that both generated transitions and external decisions operate on the same unified structure. This preserves determinism while enabling context-aware behavior without introducing separate semantic passes.

Example:

```
switch (state) {  
  case STATE_IDENTIFIER:  
    if token == Identifier:  
      symbol = extern resolveSymbol(token)  
  
    if symbol.isType:  
      transition SHIFT to STATE_TYPE_REF  
    else if symbol.isAlias:  
      transition SHIFT to STATE_ALIAS_REF  
    else:  
      transition SHIFT to STATE_VAR_REF  
  else:  
    transition ERROR  
}
```

In this example, the parser reaches a shared state (*STATE_IDENTIFIER*) that is used by multiple syntactic constructs. The external anchor (*resolveSymbol*) performs

semantic analysis, such as symbol lookup within the current scope.

The result of this external evaluation determines the subsequent transition, effectively guiding the parser into the correct branch without requiring additional lookahead or backtracking.

Because this decision point is merged across all relevant rules, the external logic operates on a unified entry point, ensuring that no duplicate states or divergent paths are introduced

7.11 Deterministic Execution Guarantees

The generated parser is fully deterministic:

- All ambiguities are resolved during generation
- No backtracking is required
- Each state has a well-defined transition for each token

This ensures predictable performance and efficient execution

8. Evaluation and Comparison

This chapter evaluates the proposed parsing approach in the context of practical usage and compares it with established techniques. The goal is not to present the system as a universally superior solution, but to position it within a set of trade-offs that proved effective in real-world scenarios

8.1 Evaluation Criteria

The evaluation is based on the following criteria:

- Execution performance
- Determinism of parsing behavior
- Flexibility and extensibility
- Integration of custom logic
- Maintainability and transparency of generated code

These criteria reflect typical requirements in tooling and compiler-adjacent systems

8.2 Comparison with Classical LR Parsers

Classical LR parsers, including LR(1) and LALR, rely on lookahead and table-driven execution. Ambiguities are resolved through parsing tables, often requiring grammar refactoring or increased lookahead.

In contrast, the presented approach eliminates ambiguities during generation through rule expansion and restructuring. Instead of resolving conflicts at runtime, the grammar is transformed until a deterministic decision structure is obtained.

This results in:

- Explicit control over parsing behavior
- No runtime conflict resolution
- A direct mapping from grammar to executable logic

However, this shifts complexity from runtime to generation time, requiring more involved preprocessing of rules

8.3 Comparison with Parser Generators

Parser generators such as ANTLR provide high-level grammar definitions and automatically generate parsers. These systems are powerful and widely used but often abstract away the execution model.

The presented approach differs in that it exposes the execution structure directly:

- Generated code closely resembles the runtime behavior (e.g., switch-based state machines)
- Developers retain full control over optimization and integration
- Custom logic can be embedded without leaving the parsing model

This transparency can be advantageous in systems where tight integration and predictable behavior are required. At the same time, it requires a deeper understanding of parsing mechanics compared to declarative tools

8.4 Comparison with Regex-Based Approaches

Regular expressions are effective for simple tokenization tasks but are inherently limited when dealing with nested or context-sensitive structures.

In particular, regex-based approaches are not suitable for:

- Macro expansion
- Conditional parsing
- Deeply nested language constructs

These limitations motivated the transition from regex-based solutions to a structured parsing approach in the described system. By introducing explicit states and transitions, the parser can handle complex language features while remaining deterministic

8.5 Performance Characteristics

The generated parser operates as a deterministic state machine, resulting in predictable and efficient execution.

Key characteristics include:

- Linear processing of input streams
- No backtracking or runtime ambiguity resolution
- Reduced branching through path merging and shared prefixes

In practice, the switch-based structure proved sufficiently fast for complex parsing tasks, including large-scale codebases. Performance was further improved through caching strategies, where initial parsing results are stored and only modified files are re-evaluated.

This approach was particularly effective in environments with isolated compilation units, where changes are localized and incremental updates can be applied efficiently

8.6 Practical Applications

The approach has been applied in several contexts, including:

- Configuration parsing (e.g., JSON-based systems)
- Build tool analysis and linting

- Preprocessor implementations for high-level languages
- Dependency resolution and project classification

In these scenarios, the combination of deterministic parsing and integrated custom logic enabled robust handling of both syntactic and context-sensitive requirements

8.7 Limitations

Despite its advantages, the approach has several limitations:

- Increased complexity in the generation phase
- Higher initial effort for grammar modeling and rule design
- Less suitable for rapid prototyping compared to declarative parser generators
- Limited out-of-the-box tooling compared to established ecosystems

Additionally, the approach relies on careful design of rules and transformations to maintain determinism, which may not always be straightforward

9. Summary and Conclusion

This paper introduced a hybrid parsing framework that combines generated deterministic state machines with manually integrated logic. The system is built around a streaming architecture with explicit tokenizer and parser separation, unified through a shared execution model based on

state stacks and transition-driven control flow.

A domain-specific language (DSL) was defined to describe grammar rules in a self-contained and extensible manner. The DSL enables rule normalization, structural expansion, and the integration of generator-defined constants. Instead of relying on runtime conflict resolution, ambiguities are eliminated during generation through rule expansion and restructuring, resulting in a deterministic execution model.

Additional mechanisms such as fragment resolution, path merging via acyclic graph construction, and directional binding of constants contribute to both performance and clarity. The integration of external logic through anchors allows context-sensitive processing without breaking determinism or introducing separate execution paths.

The presented approach demonstrates that parsing systems can benefit from a tighter integration between generated and manual logic while maintaining a deterministic execution model. By shifting complexity from runtime to generation time, the framework avoids common issues such as backtracking, ambiguous transitions, and opaque execution behavior.

In practice, the system proved sufficiently performant for large-scale projects, particularly when combined with incremental processing and caching strategies. Its explicit and transparent structure makes it well-suited for environments where control over execution, integration with existing systems, and predictable behavior are more important than minimal setup or declarative simplicity.

At the same time, the approach requires careful rule design and introduces

additional complexity in the generation phase. It is therefore best applied in scenarios where these trade-offs are justified by the surrounding system requirements.

Several areas for further development can be identified:

- **Automation of Grammar Expansion:** While the paper formalizes expansion and conflict resolution, further work could focus on fully automating these transformations in a generalized and reusable manner
- **Advanced Optimization Strategies:** Extending graph-based optimizations beyond prefix merging, such as deeper state minimization or cross-rule merging, may further improve performance and reduce generated code size
- **Tooling and Visualization:** Developing tools to visualize generated state machines and rule transformations could improve usability and reduce the complexity of debugging and maintenance
- **Incremental and Parallel Parsing:** Building on the existing streaming model, future work could explore more advanced incremental or parallel parsing strategies for large-scale systems
- **Integration with Semantic Analysis Pipelines:** The external anchor mechanism provides a foundation for deeper integration with semantic processing. Expanding this into a more formalized interface between syntax

and semantics could further broaden the applicability of the framework

Overall, the hybrid approach outlined in this paper provides a flexible foundation for building high-performance, domain-specific parsing systems, with clear opportunities for refinement and extension